AI and Society · SC3098 · Group Design Project

Working with Financial Data

From prices to returns to rebalancing: the arithmetic your platform must get right

Joel Quek · Lecturer, CCDS

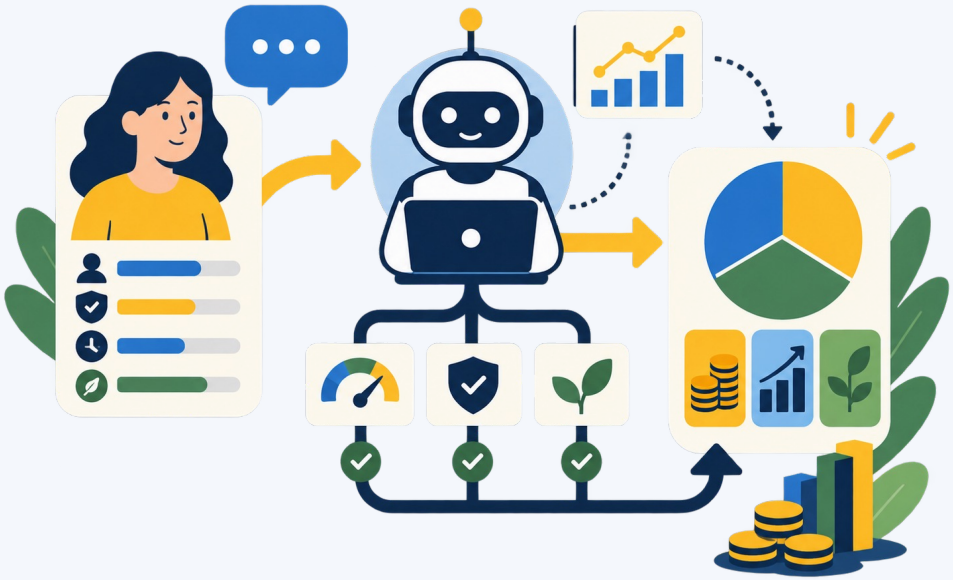
College of Computing and Data Science · Nanyang Technological University

Tuesday lab · Software Project Lab · companion Colab: SC3098_Rebalancing_Lab.ipynb

So what is the robo-advisor actually doing?

Recap: not predicting the market, but matching a person to an allowed portfolio

Ask	Who are you?
Assess	What risk fits?
Match	What is allowed?
Invest	Put money to work
Watch	Revalue and adjust



The “robo” part is the automation of the advisory workflow, not a robot that knows the future.

Three building blocks, three different jobs

Recap: the project's core portfolios mix SPY, AGG and BIL

SPY

Broad equity ETF

Role: long-term growth

Main risk: market drawdowns

AGG

Broad bond ETF

Role: income + stabiliser

Main risk: rate / credit moves

BIL

T-bill ETF

Role: liquidity + safety

Main risk: inflation / reinvestment

None of these is “the best asset”. They are useful for different reasons.

Core and satellite: broad first, concentrated second

Recap: the satellite is optional exposure, not a replacement for diversification

CORE · the bulk of the portfolio

Broad holdings that do most of the work. In this project: SPY + AGG + BIL. The point is breadth, stability of design, and a clear baseline allocation.

SATELLITE · small, capped, optional

Concentrated sector or thematic exposure, taken in small amounts on top of the core. These are also ETFs, but each tracks a single industry, so they swing harder than the broad market.

The four satellite ETFs

XLK	Technology
XLV	Healthcare
XLE	Energy
SMH	Semiconductors

Why cap it?

Concentration risk is real, and access is gated behind a knowledge check.

Now we can finally define a portfolio

Recap: a portfolio is a collection of investments and the weights assigned to them

Example: $50,000 in the project's Balanced target



The weights are the key idea.

Yesterday we promised the lab would turn these dollar targets into units, revalue them as prices move, and show how the weights drift. That is exactly what today does.

WARM-UP

Four questions before we start

1. Name the three core ETFs and the job each one does.
2. A client asks: "what score did I get?" What may the chatbot say?
3. Why is satellite exposure capped and gated?
4. Your client can tolerate big losses but needs half the money next year. What wins?

Today: the numbers behind checklist A.5 and A.6

Every formula on these slides is one your platform must compute and a supervisor will verify

Prices	Returns	Units & value	Drift	Rebalance
What the CSV gives you, and which column to trust	Simple, log, and holding-period: how value changes over time	Turning $50,000 into holdings, and holdings into value	Why weights wander away from target on their own	The trades that bring them back, and the check that they did

Everything runs on the instructor price data (Appendix C) and the Balanced portfolio (Appendix E). The companion Colab walks the same numbers in code, and the worked example on these slides matches it exactly.

The data you have

One row per ticker per trading day (Appendix C)

date	ticker	open	close	adj_close	volume
2026-02-27	SPY	...	...	682.36	...
2026-03-02	SPY	...	...	682.75	...
2026-03-02	AGG	...	...	98.99	...
2026-03-02	BIL	...	...	90.06	...

(high and low omitted here for space; the file has them)

close vs adj_close

close is the traded end-of-day price. adj_close folds dividends and splits back into the price path, so returns computed from it are total returns, as if every distribution were reinvested. All platform valuations use adj_close. This is a simulation convention: a real brokerage executes at market prices and accounts for distributions separately.

Why this matters

AGG pays monthly distributions. Compute returns from close and you silently throw that income away, understating bond returns all semester. One nuance: units priced off adj_close are total-return units, not literal share counts.

Watch for this one.
Computing returns from close where adj_close is required is the most common arithmetic bug. The demos check against adj_close.

Simple returns

How much did one dollar become, from one trading day to the next?

$$r_t = \frac{P_t}{P_{t-1}} - 1$$

Worked, from your own price file:

	SPY adj_close
2026-02-27	682.36
2026-03-02	682.75
r = 682.75 / 682.36 - 1	= 0.0006 (0.06%)

Read it as money

r = +0.0006 means $1 became $1.0006 over the next trading period (here that period spans a weekend). Multiply by a position value and you get the dollar change. Simple returns are the language of money.

One period only

A simple return describes one step. To chain many steps together you multiply growth factors, which is the next slide.

Compounding

Growth factors multiply; returns do not add

$$1 + r_{\text{total}} = (1 + r_1)(1 + r_2)\cdots(1 + r_n)$$

The classic trap

+50% then -50% does not average to zero: 1.5 x 0.5 = 0.75, so you are down 25%. Averaging simple returns overstates growth whenever there is volatility.

The habit to build

Convert to growth factors (1 + r), multiply through, subtract 1 at the end. In pandas: (1 + r).prod() - 1, or (1 + r).cumprod() for the whole path.

Your holdings view should revalue from prices,
never by adding up daily percentage changes.

Log returns

The version analysts use when they want returns to add

$$r_t^{\log} = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Why bother: additivity

Log returns sum across time: the 3-month log return is exactly the sum of the daily ones. That makes averaging, scaling and statistics clean, which is why volatility work uses them.

Small-move intuition

For small moves they are nearly identical: ln(1.006) is 0.00598. They diverge on big moves.

Which one when

Log returns for analysis: volatility, averages over time, statistical work.

Simple returns for money: valuing holdings, client reporting, anything a dollar sign touches.

One rule

Never mix them in one calculation, and never show a client a log return. Your platform's client-facing numbers are simple returns, full stop.

Holding period return

One number for the whole time an investment was held

$$\text{HPR} = \frac{V_{\text{end}} - V_{\text{start}}}{V_{\text{start}}} = \frac{P_{\text{end}}}{P_{\text{start}}} - 1$$

Worked: SPY held 2026-03-02 to 2026-06-01

	adj_close
2026-03-02	682.75
2026-06-01	756.59
HPR	= 0.1081 (+10.81% over 63 trading days)

Annualising, carefully

(1 + HPR)^(252/63) - 1 restates it as a yearly rate. Fine for comparison; dangerous as a promise. The disclosure block exists precisely because past rates do not repeat.

Your dashboard's per-holding performance figure is an HPR: value now against cost, using adj_close throughout.

From dollars to units

The exact arithmetic of checklist A.5 (Appendix C, section C.5)

$$\text{units}_i = \frac{w_i \times \$50{,}000}{P_i^{\text{adj}}(D_0)}$$

P adj is the adj_close column; units are recorded to 4 decimal places.

Balanced portfolio (55/35/10), session date 2026-03-02:

Ticker	Target w	Allocation	adj_close at D0	Units (4 dp)
SPY	55%	$27,500.00	682.75	40.2782
AGG	35%	$17,500.00	98.99	176.7823
BIL	10%	$5,000.00	90.06	55.5200

Units are fractional by design.
This is how real robo-advisors work: dollar-based investing needs fractional holdings to track target weights with negligible residual. Portfolio value at any date = sum of units x that date's adj_close.

Weights drift on their own

Same units, three months later: nobody traded, the weights moved

Ticker	Units	adj_close at 2026-06-01	Value	Actual w	Target w
SPY	40.2782	756.59	$30,474.12	57.67%	55%
AGG	176.7823	98.01	$17,326.19	32.79%	35%
BIL	55.5200	90.86	$5,044.62	9.55%	10%
Total			$52,844.92	100%	100%

Equities rallied, so SPY grew from 55% to 57.67% of the portfolio: the client is now carrying more equity risk than they agreed to. Drift is not a bug; it is what prices do. Detecting it is why the holdings view compares actual against target.

Checklist A.5 demo 4:
log in at a later session date, show holdings revalued and actual weights drifting from target.

Two backtests of the same portfolio

Both claim to track $50,000 in 55/35/10 from the first day of the file. Discuss.

```python
# Method A: "just weight the returns"
w = np.array([0.55, 0.35, 0.10])
rets = px.pct_change().dropna()
port_r = (rets * w).sum(axis=1)
wealth_A = 50_000 * (1 + port_r).cumprod()
```

```python
# Method B: buy units, hold them
w = np.array([0.55, 0.35, 0.10])
units = 50_000 * w / px.iloc[0]
wealth_B = (px * units).sum(axis=1)
# no trades after day one
```

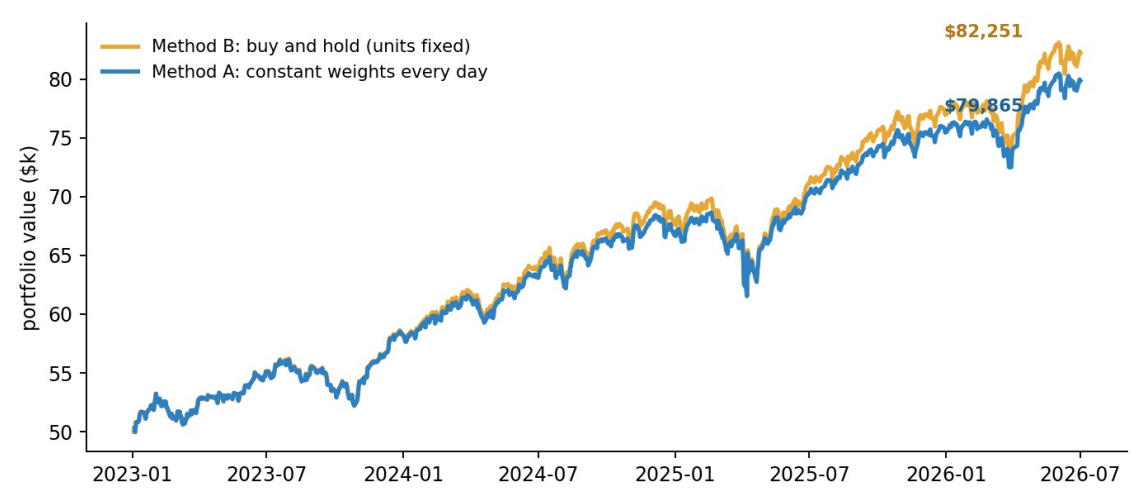
Question for the room:

after 876 trading days on the same price file, do wealth_A and wealth_B end at the same number? If not, which is higher, and what is Method A quietly assuming?

Hint: Method B never trades after day one. Does Method A?

They differ by $2,385. Method A trades every day.

Same file, same weights, same $50,000: two different strategies



Method A assumes the same weights every day

Multiplying every day's returns by 55/35/10 is only valid if the portfolio holds those weights every day. Keeping weights constant while prices move requires trading back to target daily, for free.

Method B is what actually happens

Units are fixed; weights drift. In this bull run the portfolio drifts toward SPY (68/24/7 by the end), so buy and hold ends higher: $82,251 vs $79,865.

The bug hides in short windows

Over our four-month lab window the gap is only $15.62. Run a short test, conclude the shortcut is fine, ship it, and the error compounds for years.

Neither line is wrong.
They answer different questions. Your platform must be Method B between rebalances, and only jump to target weights when the client presses the button.

Two risk numbers worth knowing

Quick definitions; the Colab computes both

Volatility

The standard deviation of daily returns, usually annualised by multiplying by sqrt(252). It measures how much the value bounces around. Computed on log returns by convention.

Maximum drawdown

The worst peak-to-trough fall over the period: how bad it got for someone who bought at the top. This is the number that makes loss-tolerance bands (Appendix B) concrete.

These two justify the risk categories: SMH's volatility and drawdowns dwarf AGG's, which is why satellites are capped and gated. You do not need either for A.5 or A.6 sign-off, but they make your explanations credible (C.3).

Rebalancing: the arithmetic

Restore target weights at today's prices (checklist A.6)

$$\text{trade}_i = \frac{w_i \times V_{\text{now}} - \text{value}_i}{P_i^{\text{adj}}(\text{now})}$$

1. Revalue

Compute V_now from units and today's adj_close.

2. Target dollars

Each instrument should hold w_i x V_now.

3. Trade the gap

Positive = buy units, negative = sell. Executed at today's prices.

Sells fund buys: net cash flow is zero (within rounding).
No money enters or leaves; the portfolio only changes shape. Your audit record for the rebalance should show this.

Automatic scheduled rebalancing and drift alerts are out of scope this iteration: rebalance runs on demand.

The worked rebalance

Same portfolio, 2026-06-01: three trades restore 55/35/10 within rounding tolerance

Ticker	Value now	Target value	Gap	Trade (units)	After: w
SPY	$30,474.12	$29,064.71	-$1,409.41	-1.8628	55.00%
AGG	$17,326.19	$18,495.72	+$1,169.54	+11.9330	35.00%
BIL	$5,044.62	$5,284.49	+$239.87	+2.6400	10.00%

Sell 1.8628 units of SPY; buy 11.9330 AGG and 2.6400 BIL. Post-trade weights hit the target to the displayed precision, and the net cash flow is $0.03, pure 4-decimal rounding residue. That residue is why A.6 says "within rounding tolerance".

A.6 sign-off:
drifted weights return to target (demo 1); trade quantities arithmetically correct against the price file (demo 2); a different session date produces different, correct trades (demo 3).

Fractional units vs whole shares

Why the simulation divides, and what real brokers changed

Your platform: fractional (divide)

units = allocation / price, to 4 dp. Fractional units get extremely close to target with negligible residual cash (we accept four-decimal rounding tolerance), and there is a single correct answer a supervisor can verify. This is also how actual robo-advisors (Betterment, StashAway, Endowus) hold client assets.

Whole shares: an optimisation problem

If you may only buy integer shares, you cannot hit 55/35/10 exactly: leftover cash appears and weights start life off-target. Choosing the best integer allocation is a small optimisation problem, and different algorithms give different valid answers.

The Colab's extension section uses PyPortfolioOpt's DiscreteAllocation to solve the whole-share version and measures the drift it introduces on day one. Instructive, and exactly why robo-advisors engineered fractional ownership. Your platform build stays fractional.

How real platforms pull this off

One common arrangement; custody details vary by platform, the ledger idea does not

Exchange / custodian: whole shares only

In an omnibus arrangement (Betterment describes its custody this way), the broker holds one pooled position per ticker. For 1,000 clients needing 40,278.2 SPY between them, it buys 40,279 whole shares.

aggregate client orders, round once at the top

The firm's internal ledger: plain database rows

client_id	spy_units
C0001	40.2782
C0002	25.1104
C0003	8.9310
... 997 more rows ...	
total client claims	40,278.2

The 0.8-share remainder is carried at the pool level: under one share per ticker, however many clients.

Daily netting

In pooled arrangements, clients buying $3.2M and selling $3.1M of SPY produce one net order at the exchange. Everyone else just swapped ledger rows.

Custody varies

Not every platform pools. Endowus describes client assets held in segregated custodian accounts in each client's name. The constant: fractional entitlements are ledger records; whole-share limits bind only where orders meet the exchange.

Where your build sits

Your platform is the ledger layer. A database of fractional units is where robo-advisors do their client accounting, whatever the custody model.

Hands-on

Open the Jupyter Notebook

SC3098_Rebalancing_Lab.ipynb, on NTULearn. Download the price CSV first; the notebook asks for it. The next slides walk the key cells; run along as we go.

The load cell

Long CSV in, wide price matrix out

```python
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

CSV = "prices_2026-07-01.csv"
if not os.path.exists(CSV):
    try:
        from google.colab import files          # running in Colab: prompt an upload
        print(f"Upload {CSV} (from NTULearn):")
        uploaded = files.upload()
        CSV = list(uploaded)[0]
    except ImportError:
        raise FileNotFoundError(f"Place {CSV} in the same folder as this notebook.")

raw = pd.read_csv(CSV)
prices = raw.pivot(index="date", columns="ticker", values="adj_close").sort_index()
prices.index = pd.to_datetime(prices.index)
print(f"{prices.shape[0]} trading days, {prices.shape[1]} tickers, "
      f"{prices.index[0].date()} to {prices.index[-1].date()}")
prices.tail(3)
```

What to notice

Colab or local, same cell

If the file is not found, the cell falls back to a Colab upload prompt. Run it and pick the CSV you downloaded.

Long to wide

The CSV has one row per (date, ticker). pivot turns it into one column per ticker, one row per day: the shape everything else wants.

Dates become real dates

pd.to_datetime on the index makes date slicing like prices.loc[D0:D1] work.

Returns and the HPR check

The formulas from the first half of the deck, in four lines each

```python
rets    = prices.pct_change().dropna()            # simple returns
logrets = np.log(prices / prices.shift(1)).dropna()  # log returns

rets[["SPY", "AGG", "BIL"]].tail(3).round(6)
```

```python
# Holding period return, and the log-additivity check
D0, D1 = "2026-03-02", "2026-06-01"          # two published session dates

hpr_spy = prices.loc[D1, "SPY"] / prices.loc[D0, "SPY"] - 1
print(f"SPY HPR {D0} -> {D1}: {hpr_spy:.4%}")

# log returns strictly after D0, up to and including D1, must sum to ln(1 + HPR)
seg = logrets.loc[D0:D1, "SPY"].iloc[1:]
assert np.isclose(np.exp(seg.sum()) - 1, hpr_spy), "log returns did not sum to the HPR"
print(f"sum of {len(seg)} daily log returns = {seg.sum():.6f} = ln(1 + HPR)  [check passed]")
```

What to notice

pct_change is the simple return

One pandas call gives P_t / P_(t-1) - 1 for every ticker at once.

Log returns via shift

np.log(prices / prices.shift(1)): same idea, additive across time.

The assert is the lesson

The 63 daily log returns sum to the log HPR; exponentiating that sum and subtracting 1 reproduces the HPR. If your implementation fails this check, fix it before anything downstream.

Investing the $50,000

Checklist A.5: dollars to fractional units, once, at 4 dp

```python
V0 = 50_000
TARGET = {"SPY": 0.55, "AGG": 0.35, "BIL": 0.10}     # Appendix E portfolio 4 - Balanced
TICKERS = list(TARGET)

p0 = prices.loc[D0, TICKERS]
units = pd.Series({t: round(V0 * w / p0[t], 4) for t, w in TARGET.items()})

holdings = pd.DataFrame({
    "target_w": pd.Series(TARGET),
    "adj_close_D0": p0,
    "units": units,
    "value": (units * p0).round(2),
})
holdings["actual_w"] = holdings["value"] / holdings["value"].sum()
print(f"invested: ${holdings['value'].sum():,.2f} of ${V0:,}")
assert abs(holdings["value"].sum() - V0) < 1.0, "more than $1 of cash left uninvested"
assert np.allclose(holdings["actual_w"], list(TARGET.values()), atol=1e-4), "weights off target at inceptio
holdings.round(4)
```

What to notice

Round once, at creation

Units are rounded to 4 dp the moment they are bought, and never again. Rounding later values instead changes the answer.

Weights verified at inception

The asserts check the invested total is within $1 of $50,000 and weights match target to 1e-4. Your platform should make the same checks.

This table is the demo

A.5 demo 2 is literally: show this holdings table in your database.

Drift

Checklist A.5 demo 4: same units, later date, moved weights

```python
p1 = prices.loc[D1, TICKERS]
value_now = units * p1
V1 = float(value_now.sum())
actual_w = value_now / V1

drift = pd.DataFrame({"target_w": pd.Series(TARGET), "actual_w": actual_w.round(4),
                      "value": value_now.round(2)})
print(f"portfolio value {D0} -> {D1}: ${V0:,} -> ${V1:,.2f}")
assert actual_w["SPY"] > TARGET["SPY"], "expected SPY to have drifted above target in this window"

drift[["target_w", "actual_w"]].plot(kind="bar", figsize=(7, 3.5),
    title=f"Actual vs target weights at {D1} (no trades since {D0})")
plt.axhline(0, color="k", lw=0.5); plt.show()
drift
```

What to notice

Units never change here

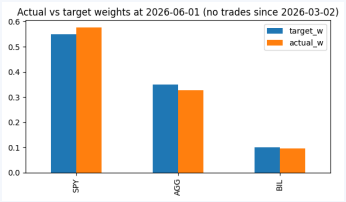
Only prices moved. Revaluation is units times the new adj_close: no trades, no cash flow.

The chart is the story

Actual weights against target at 2026-06-01: SPY has grown to 57.7%. This is the drift the client sees on their dashboard.

An assert with a view

The cell asserts SPY drifted above target in this window, so a broken revaluation fails loudly.



The rebalance

Checklist A.6: the trades, the two checks, and the audit record

```python
target_value = pd.Series({t: w * V1 for t, w in TARGET.items()})
trades = ((target_value - value_now) / p1).round(4)      # units to buy (+) or sell (-)

new_units = units + trades
new_w = (new_units * p1) / V1
net_cash = float((trades * p1).sum())

rebalance = pd.DataFrame({"value_now": value_now.round(2), "target_value": target_value.round(2),
                          "trade_units": trades, "new_w": new_w.round(4)})
print(f"net cash flow of the rebalance: ${net_cash:+.2f}  (pure 4 dp rounding residue)")
assert np.allclose(new_w, list(TARGET.values()), atol=1e-4), "rebalance failed to restore target weights"
assert abs(net_cash) < 1.0, "rebalance created or destroyed more than $1"

audit_record = {                                  # the shape of an A.6 audit entry
    "action": "rebalance", "session_date": D1, "portfolio": "4 - Balanced",
    "pre_units": units.round(4).to_dict(), "trades": trades.to_dict(),
    "post_units": new_units.round(4).to_dict(),
    "value": round(V1, 2), "net_cash_flow": round(net_cash, 2),
}
audit_record
```

What to notice

One formula, three trades

(target dollars minus current dollars) over price, rounded to 4 dp: sell SPY, buy AGG and BIL.

Two properties or it did not happen

Post-trade weights equal target within 1e-4, and net cash flow is under $1. Both are asserted; both are what the supervisor checks.

The audit dict is a template

Copy its shape for your platform's audit log: pre-units, trades, post-units, value, net cash.

The whole-share detour

Optional extension: what changes if you may only buy integer shares

```
# %pip install PyPortfolioOpt -q     # <- uncomment and run once in Colab, then re-run this cell
try:
    from pypfopt.discrete_allocation import DiscreteAllocation
    HAVE_PYPFOPT = True
except ImportError:
    HAVE_PYPFOPT = False
    print("PyPortfolioOpt not installed; uncomment the pip line above and run again. Skipping.")

if HAVE_PYPFOPT:
    da = DiscreteAllocation(TARGET, p0, total_portfolio_value=V0)
    shares, leftover = da.greedy_portfolio()
    int_value = pd.Series({t: shares.get(t, 0) * p0[t] for t in TICKERS})
    int_w = int_value / int_value.sum()
    compare = pd.DataFrame({"target_w": pd.Series(TARGET),
                            "fractional_w": holdings["actual_w"].round(4),
                            "whole_share_w": int_w.round(4),
                            "whole_shares": pd.Series(shares)})
    print(f"leftover cash under whole shares: ${leftover:,.2f}  (fractional: $0.00)")
    display(compare)
```

What to notice

Whole shares as an optimisation

DiscreteAllocation picks integer share counts: $224.14 of cash is left over and weights start off target from day one. The contrast is the fractional-units argument made concrete.

Then the exercises

Growth portfolio, different dates, a drift loop. Each has an assert, so you know when you are done.

THE ONE IDEA TO KEEP

Prices change; units do not, until you trade.

Value = units x adj_close. Drift is what prices do to weights; rebalancing is the one deterministic trade that undoes it.

Run the notebook before next week's lab.